

Antar — Technical Walkthrough

Interview reference · Week 1 state · Blocks C and D Repo: github.com/baratwajsucharitha-creator/Capstone_Project

The framing that matters

You did not build a calculator. You built a domain model with an audit trail.

Every design decision in this document defends that sentence. If you remember nothing else, remember that a calculator returns ₹1,23,825 and a product tells the family *why* — and that the difference is architectural, not cosmetic.

The 30-second opening

"It's a Maven multi-module Java 17 project. The `antar-engine` module is pure Java with test-scope dependencies only — no Spring, no framework. It holds an immutable domain model of a health insurance policy and a hospital bill, and a deterministic rules engine that computes what the insurer would actually pay. Every deduction it applies emits a trace record naming the policy clause that authorised it, and a test asserts the traces sum exactly to the gap. Week 2 wraps it in Spring Boot; the engine itself never learns Spring exists."

That paragraph contains five hooks — multi-module, framework independence, immutability, determinism, the trace invariant. Say it, then stop. Let them pick one.

Part 1 — Project structure

```
antar/
├─ pom.xml                parent, packaging=pom
├─ antar-engine/
│   ├─ pom.xml            JUnit + AssertJ, test scope only
│   └─ src/main/java/com/antar/engine/
│       ├─ model/         11 records and enums
│       ├─ GapEngine.java
│       └─ DomainModelDemo.java
```

Why multi-module when there is only one module?

Because `antar-api` arrives in Week 2, and the moment it does, the dependency direction must be enforced by the build rather than by discipline. `antar-api` will depend on `antar-engine`. The reverse is impossible — the engine has no compile-scope dependencies at all.

Why that matters

Business rules that depend on a framework can only be tested by starting that framework. The engine tests run in milliseconds with no Spring context, no database, no HTTP.

That is a hexagonal-architecture argument without needing to say "hexagonal architecture." Say the concrete version — it lands better than the vocabulary.

The parent POM

Does dependency management (JUnit BOM imported, versions in one place) and sets `maven.compiler.release=17`.

`release` rather than `source`/`target` — it validates against the Java 17 **API surface**, so you cannot accidentally call a Java 21 method while claiming to target 17. `source`/`target` only check syntax.

Part 2 — The domain model, class by class

Money

The one everyone asks about.

```
java

public record Money(BigDecimal amount) implements Comparable<Money> {
    public Money {
        if (amount.signum() < 0) throw new IllegalArgumentException(...);
        amount = amount.setScale(2, RoundingMode.HALF_UP);
    }
}
```

What it is: a value object wrapping `BigDecimal`.

Why not `double`: binary floating point cannot represent 0.1 exactly. Money arithmetic accumulates error, and in a system whose entire output is a rupee figure, that is fatal. Say it plainly — *"There is no `double` anywhere in the calculation path."* Interviewers notice.

Why not `BigDecimal` directly: three reasons.

1. It permits negative amounts
2. It permits inconsistent scales — `100.0` and `100.00` are unequal under `equals()`
3. It carries no domain meaning; a `BigDecimal` parameter could be an amount, a percentage, or a day count

The compact constructor is the interesting part. Records run it before field assignment, so validation and normalisation happen at construction time. **An invalid `Money` cannot exist.**

That is the argument to make — not "I validate inputs" but "the type system makes the invalid state unrepresentable."

`ratioTo()` returns scale 6, deliberately

The room proportion here is $5000/8000 = 0.625$, which is exact. But a 1% limit against a ₹7,000 room gives 0.714285...

If you round that to two places before multiplying ₹1,90,000, you shift the answer by hundreds of rupees. So proportion factors keep six decimal places; only the resulting **amount** rounds to two.

If precision comes up, this is the example to give. It shows you thought about *where* rounding happens, not merely *that* it happens.

Why `implements Comparable`

So `min()` and `isGreaterThan()` read as domain language.

```
java
payable.min(cap) // says what it means
payable.amount().compareTo(cap.amount()) <= 0 // does not
```

`BillCategory`

Behaviour, not labels.

```
java
public enum BillCategory {
    ROOM(false), ASSOCIATED(true), PHARMACY(false),
    IMPLANT(false), NON_PAYABLE(false);
    private final boolean proportionedByDefault;
}
```

The insight: "Nursing charges – 2nd floor ward" is a hospital's free text. `ASSOCIATED` is a *behaviour under a policy*. The engine switches on the category, never on the description string.

Why that is defensible: hospital bills have no standard vocabulary. One hospital writes "OT Charges", another "Operation Theatre Utilisation". Any engine matching on text breaks the moment it meets a new hospital.

Classification is a separate concern — and eventually the LLM's job in Week 6. The engine consumes a category and stays stable.

The enum carries `isProportionedByDefault()` because the default belongs **with** the category, not scattered through the engine.

`BillLine` **and** `HospitalBill`

`BillLine` is a validated triple: description, category, amount.

`HospitalBill` is the aggregate — procedure code, days admitted, actual room rent per day, and the lines.

Three things to point out

`List.copyOf(lines)` **in the compact constructor**. Records give you a shallow-immutable *reference*. Without the copy, the caller retains a handle on a mutable list and can change your "immutable" bill after construction. `List.copyOf` returns an unmodifiable copy, so the aggregate is genuinely immutable.

`amountIn(category)` **uses a stream reduction** over `Money::plus` with `Money.ZERO` as identity. Clean, and it means the engine never writes summation loops.

`actualRoomRentPerDay` **is a separate field**, not derived from `ROOM ÷ days`. Real bills have variable room rates — three days in a ward, two in ICU. The per-day rate driving eligibility is a distinct fact from the total charged. Modelling it separately is what lets Week 5 add ICU sub-limits without restructuring anything.

`RoomRentRule`, `SubLimit`, `CoPayRule`, `CoPayOrder`

The clause types — the policy's terms expressed as data.

`RoomRentRule`

Holds `percentOfSumInsuredPerDay` (0.01 for "1% of SI per day") plus `proportionPharmacy` — because a minority of insurers **do** proportion pharmacy, and hardcoding "they don't" would be wrong for those products.

`CoPayRule`

Two details worth flagging:

`retainedFactor()` **returns** `1 - percent`, so the engine multiplies by 0.90 rather than computing a 10% deduction and subtracting it. Fewer operations, less rounding.

`CoPayRule.NONE` **is a null-object constant**. The engine never null-checks a co-pay rule. A policy without co-pay carries `NONE`, whose `applies()` returns false.

`CoPayOrder` — **talk about this one longest**

```
java
```

```
public enum CoPayOrder { BEFORE_PROPORTION, AFTER_PROPORTION }
```

Two values, no methods, and it is **the most important design decision in the model**.

Insurers genuinely disagree about whether co-pay lands before or after the room-rent proportionate deduction — and the order changes the final rupee figure.

Most people building this would hardcode one order, because whichever policy they tested against did it that way. Making it an explicit enum on the policy terms says: *the variation is real, I found it, and I modelled it rather than assuming it away*.

This is the single strongest signal in the codebase that you did domain analysis rather than arithmetic.

PolicyTerms and **Policy**

PolicyTerms

Composes the clauses and exposes `subLimitFor(procedureCode)` returning `Optional<SubLimit>`.

`Optional` as a **return type** is exactly its intended use, and it forces the engine to handle absence explicitly rather than risk a null.

Policy

Holds `sumInsured` and `remainingSumInsured` as separate fields, with an invariant that remaining can never exceed the total.

Why two fields: in a family floater, cover is shared. A claim by one member reduces what is available to everyone else. `remainingSumInsured` is **state that moves over time**, not a policy constant.

Modelling it separately is what makes the scenario "*your father's claim in March changed your cover in September*" expressible at all.

DeductionTrace

The one that makes it a product.

```
java
```

```
public record DeductionTrace(String clauseReference, String explanation, Money
    public DeductionTrace {
        if (clauseReference == null || clauseReference.isBlank())
            throw new IllegalArgumentException("every deduction must name a cla
    }
}
```

Every rupee the engine removes produces one of these. The validation refuses to construct a trace without a clause reference and a human-readable explanation.

Why this answers "what makes your project different"

A calculator returns ₹1,23,825.

A product says: ₹18,000 non-payables, ₹15,000 room excess, ₹71,250 proportionate deduction, ₹19,575 co-pay.

Same number. Completely different value.

And it is not only UX. It is testability, auditability and debuggability. When a real settlement letter disagrees with the engine, the trace shows you exactly which step diverged.

GapResult

Total bill, payout, gap, and the trace list. Plus two derived methods:

gapAsPercentOfBill() — the number that produces the reaction. "₹1,23,825" is abstract. "41% of your bill" is not.

totalDeductions() — sums the trace. This exists specifically so a test can assert it equals the gap.

Part 3 — GapEngine

One public method, six steps, no state.

1. NON_PAYABLE_ITEMS	strip IRDAI Annexure I items entirely
2. ROOM_RENT_CAP	allowed room = eligible/day × days, if actual exceeded eligible
3. ROOM_RENT_PROPORTION	associated × (eligible/actual) pharmacy + implants pass through untouched
4. CO_PAY	× retainedFactor
5. PROCEDURE_SUB_LIMIT	min(payable, cap)
6. SUM_INSURED_EXHAUSTED	min(payable, remaining)

gap = totalBill - payout

It is stateless and deterministic

Same bill, same policy, same rupee — always. No randomness, no clock, no LLM in the calculation path.

That is what makes it testable, and what makes it trustworthy for something people will make financial decisions on.

Step 3 is the product insight

The room-rent cap costs ₹15,000. The proportionate deduction it **triggers** costs ₹71,250 — nearly five times more.

Choosing a room one grade up silently reduces the surgeon's fee reimbursement. Families never see this coming, and the engine makes it visible as a separate trace line.

Pharmacy and implants are excluded from the proportion by default

Controlled by `roomRentRule.proportionPharmacy()`. There is a dedicated test asserting they survive — because getting this wrong is the single most common error in this domain.

Step 5 comes after co-pay deliberately

A sub-limit is an absolute ceiling on the payout, so applying it last is correct.

Being able to explain **why each step sits where it does** is the point of the whole section.

Part 4 — The tests

Six tests. Each defends a specific claim.

Test	What it proves
Room within limit	Baseline — no proportion applied when it should not be
Room above limit + co-pay	Full pipeline verified end to end at ₹1,76,175
Pharmacy survives proportion	The most commonly-got-wrong rule
Cataract sub-limit	Absolute caps override the computed payout
Sum insured exhausted	Floater state constrains the answer
Deductions sum to the gap	The invariant

Talk about the last one

```
java
assertThat(result.totalDeductions()).isEqualTo(result.gap());
```

If any future change removes money without emitting a trace, the build breaks.

It converts *"every rupee is explainable"* from a stated principle into an **enforced** one. That is the difference between a design principle and a design guarantee — and noticing that distinction is a genuinely senior thing.

Part 5 — Follow-up questions you will get

"Why records rather than classes or Lombok?"

Immutability by default. `equals`/`hashCode`/`toString` generated correctly. Compact constructors give validation at construction. No annotation processor in the build. For a domain model that is entirely value objects, records are exactly the right tool.

"How would you handle a new insurer with different rules?"

Today `PolicyTerms` covers the variation I modelled. Week 5 lifts rule ordering into versioned, effective-dated rule sets, so an insurer's terms become a data row rather than a code change — and historical claims still compute against the terms in force on the date of admission.

"What is your test coverage?"

Six scenarios today, targeting 25+ with `@ParameterizedTest` driven by a CSV of insurer variants.

Do not inflate this. Six well-chosen scenarios honestly described beats a claimed 90% that collapses under one follow-up.

"Where does the AI come in?"

Policy PDF parsing, Week 6, behind a redaction layer. It never computes.

The moment a probabilistic component touches a rupee figure, the output stops being defensible — and this is a domain where being right matters more than being clever.

"What is not done?"

`CoPayOrder.BEFORE_PROPORTION` is modelled but not yet honoured. No waiting periods, no no-claim bonus, no multi-policy stacking. All listed in the README under Known Gaps.

Answer this one straight. Volunteering your gaps accurately is a stronger signal than a longer feature list — and having them already documented in the repo proves it is not post-hoc.

The one thing to fix before you rely on this

The expected values in the tests come from the worked example, not from your own policy.

Run one real hospitalisation scenario through by hand and check the engine agrees.

Until you have done that, *"my family uses it"* is a line you cannot say — and it is the line that makes the whole project land.

Quick reference — the numbers

Sum insured	₹5,00,000
Eligible room rent	₹5,000/day
Actual room taken	₹8,000/day × 5 days
Proportion factor	0.625
Total bill	₹3,00,000
Payout	₹1,76,175
Gap	₹1,23,825 (41.3%)

Trace: non-payables ₹18,000 → room cap ₹15,000 → proportion ₹71,250 → co-pay ₹19,575